EventPerp

Leveraged perpetuals on binary outcome tokens, settled to the outcome

Version 1.0 · 30 September 2026
Builds on the build plan of 27 September 2026 (`roadmap.md`); parameter values stay in its §3
Supersedes `docs/superpowers/specs/2026-09-29-eventperp-theory-design.md`
Status design complete; five decisions open (§12)

EventPerp gives traders leveraged YES/NO exposure on event markets and pays every position against the real outcome. The hard problem is resolution: the price jumps to 0 or 1, no liquidation can act inside a jump, so leverage must reach exactly 1 before it happens. This document derives the maths from first principles, specifies an on-chain order book for Monad's page-priced storage, tests whether an off-chain/on-chain hybrid would cut latency, and lists the full technology stack.

Contents

1. Summary
locked decisions, what this document adds, notation

2. The underlying: price, variance, jumps
why leverage must fall to 1x before resolution

3. Positions and accounting
fills, conservation, Proposition 1, the price-free safety test

4. Margin and the leverage schedule
buffers, tiers, and the model that justifies them

5. Index, mark and manipulation
where prices come from and what an attacker can do

6. Funding

7. Liquidation, deleverage and withdrawals

8. Settlement

9. The order book
bitmap, FIFO levels, Monad storage layout, matching loop

10. Latency and the hybrid question
can an off-chain + on-chain book cut latency? verdict

11. Technology stack
every layer, tool and owner

12. Changes to the build plan and open decisions

13. Limitations

14. Sources

1. Summary

> **IN PLAIN WORDS**
> A YES token pays 1 USDC if an event happens and 0 if it doesn't, so its price is the market's probability. EventPerp lets you trade that price with leverage. Leverage is fine while the event is undecided, but at resolution the price jumps straight to 0 or 1 and a leveraged loser can't pay. So EventPerp offers leverage early and steps it down to 1x six hours before resolution. Everything else in this document exists to make that schedule safe, fair and fast.

1.1 Locked decisions

Area	Decision
Product	One YES/NO question per market. Positions settle to the outcome $Y \in \{0, \frac{1}{2}, 1\}$; $\frac{1}{2}$ means INVALID
Markets	Native markets (own YES/NO tokens traded on Kuru, $L_{\text{cap}} = 3$) are required. Mirrored markets (Polymarket price and outcome, $L_{\text{cap}} = 5$) are optional
Collateral	USDC only, isolated per (trader, market), no trading fees
Leverage	Measured against worst-case loss, $L = W/E$. Four announced tiers; the final tier (last 6 h) requires full collateral, checked without any price
Index	A keeper pushes the 60 s median Kuru mid; the chain applies a speed limit and a slow average
Mark	The index nudged toward the perp spread, at most $b(I)$ away; never used for liquidation
Liquidation	Needs both the fast and the slow index; the position moves to the insurance-fund account at the index and is never sold into the book
Funding	Absolute premium with an edge ramp and a collateral-based cap; frozen in the final tier
Settlement	Pull-based claims with one pro-rata ratio r for everyone
Oracle	Three LLM voters from different providers via Chainlink CRE; unanimity and confidence gate on-chain; bonded challenge; committee Safe
Order book	Fully on-chain CLOB inside the core contract: 999-tick bitmap, FIFO levels, storage laid out for Monad's pages (§9)
Chain	Monad testnet; one core contract; per-market and per-account state only

1.2 What this document adds

- **Derivations** for every formula in the build plan, and the models behind the tiers (§2–§8).
- **The order book design** for Monad's page-priced storage, live since 2 September 2026 (§9).
- **The latency question:** what an off-chain/on-chain hybrid would and would not speed up, a full hybrid design, and a recommendation (§10).
- **The full technology stack**, layer by layer (§11).
- **Fourteen changes** to the build plan and **five open decisions** (§12).

1.3 Notation

Notation follows the build plan: τ is the resolution time, t now, $T = \tau - t$ the time left.

Symbol	Meaning
$n = s\,x$	signed position ($s = +1$ long, -1 short) of size x units; one unit pays 1 USDC on YES

Symbol	Meaning
p_0, $N = n\,p_0$	average entry price; stored entry notional
C	collateral, including realised P&L, settled funding and penalties
q	mark price
I, I_S, m_k	fast index, slow index, last raw keeper sample
$w(p)$	worst-case loss per unit: p for a long, $1-p$ for a short
$W = x\,w(q)$, $E = C + n(q-p_0)$, $L = W/E$	worst-case loss, equity, leverage
$J(T)$, $e(x)$, u	jump buffer, path + execution buffer, initial-margin headroom
MM, IM	maintenance and initial margin
F, f	cumulative funding index, funding rate
B_m, IF	money held by market m, insurance-fund balance
Φ, ϕ; φ	normal CDF and density; liquidation penalty rate

2. The underlying: price, variance, jumps

> **IN PLAIN WORDS**
> A YES price is a probability that must end at exactly 0 or 1. However little time is left, it still has the same amount of "movement" to get through, so it becomes more volatile as resolution approaches. On top of that smooth movement, decisive news can make it jump in one block. Both facts force leverage down to 1x.

2.1 Binary tokens

YES pays 1 USDC if the event happens; NO pays 1 USDC if it does not. The OutcomeVault mints a complete set, 1 YES + 1 NO, for 1 USDC and redeems it for 1 USDC, so every token pair is fully backed. If YES and NO together trade above 1, arbitrageurs mint and sell; below 1, they buy and redeem. Hence the YES and NO prices add up to about 1, and **shorting YES is economically the same as buying NO**.

2.2 The price is a martingale

Let $Y \in \{0,1\}$ be the outcome and $\mathcal{F}_t$ everything known at time t. In a fair market the price is the expected payoff:

$$p_t = \mathbb{E}[Y \mid \mathcal{F}_t], \qquad p_\tau = Y \in \{0,1\}.$$

By the tower property, for any later $s > t$: $\mathbb{E}[p_s \mid \mathcal{F}_t] = \mathbb{E}\big[\mathbb{E}[Y \mid \mathcal{F}_s] \mid \mathcal{F}_t\big] = p_t$. The best forecast of tomorrow's price is today's price: there is no predictable drift, and every move is news. Discounting is ignored because markets last days to weeks and collateral earns no interest.

Example. YES if at least one of two fair coins lands heads: $p_0 = 0.75$. After the first flip $p_1 = 1$ (heads) or 0.5 (tails), and $\frac{1}{2}\cdot 1 + \frac{1}{2}\cdot 0.5 = 0.75 = p_0$.

2.3 The variance budget

Claim. The expected sum of squared price moves from now to resolution is exactly $p_t(1-p_t)$, however the time is sliced.

1. Because $Y^2 = Y$: $\mathbb{E}[(Y - p_t)^2 \mid \mathcal{F}_t] = p_t - 2p_t^2 + p_t^2 = p_t(1 - p_t)$.
2. Split the path at $t = t_0 < t_1 < \cdots < t_n = \tau$ with moves $\Delta_k = p(t_k) - p(t_{k-1})$. They telescope: $Y - p_t = \sum_k \Delta_k$.
3. Martingale moves are uncorrelated: for $j < k$, $\mathbb{E}[\Delta_j \Delta_k] = \mathbb{E}\big[\Delta_j \, \mathbb{E}[\Delta_k \mid \mathcal{F}_{t_{k-1}}]\big] = 0$.
4. Squaring the sum, every cross term vanishes in expectation:

$$\mathbb{E}\Big[\sum_{k=1}^{n} \Delta_k^2 \;\Big|\; \mathcal{F}_t\Big] = \mathbb{E}\big[(Y - p_t)^2 \mid \mathcal{F}_t\big] = p_t(1 - p_t).$$

Coin check: from $p_0 = 0.75$ the squared-move sums are 0.0625 (probability ½) and 0.3125 (probability ¼, twice), averaging $0.1875 = 0.75 \times 0.25$.

Consequence. The budget does not shrink with time. With a month left it is spread over a month; with an hour left it must be spent in an hour. Per-minute volatility therefore rises as $T \to 0$. BTC has no such deadline, which is why crypto perps can keep a fixed maximum leverage and EventPerp cannot.

2.4 Local volatility: the diffusion model

Model the event as a hidden Brownian motion finishing above a threshold, $\{B_\tau > c\}$. With $z = \Phi^{-1}(p)$:

$$p_t = \Phi\Big(\frac{B_t - c}{\sigma_B \sqrt{T}}\Big), \qquad dp_t = \frac{\phi(z)}{\sqrt{T}} \frac{dB_t}{\sigma_B} \;\Rightarrow\; \sigma_p(p, T) = \frac{\phi\big(\Phi^{-1}(p)\big)}{\sqrt{T}}.$$

The drift vanishes because p is a martingale, and σ_B cancels. Volatility is largest at $p = \frac{1}{2}$ ($\phi(0) = 0.399$), smaller near the edges, and grows like $T^{-1/2}$. A k-sigma adverse move during a liquidation delay h is

$$\Delta p_k(p, T) = k \, \phi(z) \sqrt{h/T}.$$

2.5 Jumps and the L = 1 threshold

Real markets also jump: the match ends, a result is published, a candidate withdraws. A two-branch model of the next window h:

- **Jump branch**, probability π_t: decisive information arrives; by the martingale property the price goes to 1 with probability q and to 0 with probability $1 - q$.
- **Diffusion branch**, probability $1 - \pi_t$: ordinary news moves the price by about $\sigma_d Z$, with Z standard normal.

A long loses its whole worst case W with probability $\pi_t(1 - q)$; a short with probability $\pi_t \, q$.

Theorem (L = 1 threshold). Let margin cover losses except with probability ε (Value-at-Risk at level ε). If $\pi_t(1 - q) > \varepsilon$ for a long, or $\pi_t \, q > \varepsilon$ for a short, then $\mathrm{VaR}_\varepsilon = W$: margin must equal the full worst-case loss and maximum leverage is exactly 1.

Proof. Loss never exceeds W (the price is bounded) and equals W with probability above ε, so the ε-quantile of the loss is W. ∎

Below the threshold, the margin v that leaves exactly ε uncovered solves

$$\pi_t(1 - q) + (1 - \pi_t)\,\Pr[x \, \sigma_d Z > v] = \varepsilon \;\Rightarrow\; v = x \, \sigma_d \, z_{\varepsilon'}, \qquad \varepsilon' = \frac{\varepsilon - \pi_t(1 - q)}{1 - \pi_t}.$$

Mark q	Long must be 1x when $\pi_t >$	Short must be 1x when $\pi_t >$
0.20	1.25%	5.0%
0.60	2.5%	1.7%
0.90	10%	1.1%

($\varepsilon = 1\%$.) The side betting on the favourite keeps leverage longest, because its bad outcome is the unlikely one.

It is a cliff, not a slope. Long at $q = 0.60$, $\varepsilon = 1\%$, $\sigma_d = 0.01$ per window:

π_t	ε'	$z_{\varepsilon'}$	Margin per unit	Max leverage
0.1%	0.96%	2.34	0.023	~26x
2.0%	0.20%	2.87	0.029	~21x
2.4%	0.04%	3.35	0.034	~18x
2.5%	0	∞	0.60 = W	1x

Margin is mathematically continuous but in practice jumps from about 0.03 to 0.60 near the threshold. The move to 1x is inherently abrupt, so EventPerp **schedules and announces it** (§4) instead of letting it happen by surprise, and lists only markets whose decisive moment is scheduled: sports, macro releases, elections.

3. Positions and accounting

> **IN PLAIN WORDS**
> Each account stores four numbers: position size, entry notional, collateral and a funding snapshot. Everything else (equity, leverage, safety) is computed from them. Every trade moves value from one account to another without creating or destroying any, and one simple test tells whether a position can survive any outcome.

3.1 Worst-case basis (Eq-1)

$$w(p) = \begin{cases} p & \text{long} \\ 1 - p & \text{short} \end{cases} \qquad W = x\, w(q) \qquad E = C + n\,(q - p_0) \qquad L = \frac{W}{E}$$

Leverage is measured against the capital that can actually be lost. A short of 1,000 units at $q = 0.90$ with $E = 100$ can lose at most 100, so $L = 1$, not the 9x that "position value ÷ equity" would report. With this definition a short YES and a long NO always show the same leverage, and every formula has a mirror obtained by replacing q with $1 - q$.

3.2 Orders, fills and the entry price (Eq-2)

An **order** is a request ("buy 600 at 0.51 or better"); a **fill** is an actual trade against a resting order, and one order can produce several fills at different prices. p_0 is the size-weighted average entry price. For a signed fill δ at price q_f, with $n' = n + \delta$:

Case	When	New entry p_0'	New collateral C'
Increase	$n\delta \geq 0$	$\frac{n\,p_0 + \delta\,q_f}{n + \delta}$	C
Reduce	$n\delta < 0, \lvert\delta\rvert \leq \lvert n\rvert$	p_0	$C - \delta\,(q_f - p_0)$
Flip	$n\delta < 0, \lvert\delta\rvert > \lvert n\rvert$	q_f	$C + n\,(q_f - p_0)$

A reduce locks the closed part's profit or loss into C and leaves the entry price of the remaining units unchanged. A flip closes everything, then opens the remainder fresh at q_f. Accounts store $N = n\,p_0$, not p_0: N is exact (for example $500 + 350 = 850$) while $p_0 = 0.5666\ldots$ is not.

3.3 Conservation (Eq-3)

Every fill changes an account's "virtual cash" $C - N$ by exactly $-\delta\, q_f$:

$$\begin{aligned} \text{increase:}\quad & \Delta(C - np_0) = -\big((n+\delta)p_0' - np_0\big) = -\delta q_f \\ \text{reduce:}\quad & -\delta(q_f - p_0) - \delta p_0 = -\delta q_f \\ \text{flip:}\quad & n(q_f - p_0) - (n+\delta)q_f + np_0 = -\delta q_f \end{aligned}$$

Buyer and seller use the same q_f with opposite δ, so their changes cancel. With the market balance B_m (deposits − withdrawals ± insurance transfers) and funding settled:

$$\sum_i \big(C_i - n_i p_{0,i}\big) = B_m, \qquad \sum_i n_i = 0 \;\Rightarrow\; \sum_i E_i(q) = B_m \;\text{ for every common } q.$$

Equity is virtual cash plus inventory, $E = (C - N) + nq$. The market is zero-sum at every price, including $q = Y$ at resolution. This is invariant INV-2.

3.4 Resolution: Proposition 1 and the price-free safety test (Eq-4)

At resolution the mark becomes the outcome: $E_\tau = C + n(Y - p_0)$. A long with $C = xp_0/L$ that loses ($Y = 0$) ends at $E_\tau = -xp_0(1 - 1/L)$, and a short mirrors it. In one line:

$$\text{Shortfall} = W(p_0)\left(1 - \frac{1}{L}\right) > 0 \quad \text{for every } L > 1.$$

Example. 1,000 YES at 0.60 with 5x: $C = 120$. If NO wins, $E_\tau = 120 - 600 = -480$ USDC that nobody pays.

From the current mark, $E_\tau = E + n(Y - q)$ and the worst case is $E - x\,w(q)$. So there is no bad debt at resolution **if and only if** $E \geq x\,w(q)$, that is $L(q) \leq 1$. And the mark cancels:

$$E \geq W \iff \begin{cases} C + x(q - p_0) \geq xq & \text{long} \\ C - x(q - p_0) \geq x(1-q) & \text{short} \end{cases} \iff C \geq x\,w(p_0).$$

Full collateral can be checked from stored numbers alone, with no price oracle. The engine enforces it **from the final tier onward** (INV-1), so it already holds when a halt or the resolution jump arrives; a halt alone is not the mechanism.

Corollary (L = 1 is spot). A matched long/short pair at p_0 with $C_{\text{long}} = xp_0$ and $C_{\text{short}} = x(1 - p_0)$ holds exactly x USDC, the collateral of x complete sets. Settling to the outcome is solvent by construction.

3.5 Liquidation prices, for intuition

With a flat maintenance fraction m of W and $C = x\,w(p_0)/L$:

$$p_{\text{liq}}^{\text{long}} = \frac{p_0(1 - 1/L)}{1 - m}, \qquad p_{\text{liq}}^{\text{short}} = \frac{p_0 + (1 - p_0)/L - m}{1 - m}.$$

Leverage	Long liquidated at	Distance	Short liquidated at	Distance
2x	0.316	0.284	0.789	0.189
3x	0.421	0.179	0.719	0.119
5x	0.505	0.095	0.663	0.063
10x	0.568	0.032	0.621	0.021

($p_0 = 0.60$, $m = 0.05$.) The short sits closer to liquidation because it has less capital at risk (0.40 against 0.60). A 10x position survives ordinary 10-minute drift but not a single 5-cent jump, which is why 10x is not offered. The engine does not use a flat m; §4 replaces it with jump and path buffers.

4. Margin and the leverage schedule

> **IN PLAIN WORDS**
> Margin is a safety deposit that must cover whatever can go wrong before a liquidator is allowed to act. It has two parts: a buffer for ordinary drift plus the cost of closing, and a buffer for sudden jumps. The jump buffer grows in announced steps as resolution nears, and in the last six hours it equals the whole worst-case loss.

4.1 The liquidation delay h

Liquidation is not instant. Between a real price move and the contract being allowed to liquidate:

Step	Delay	Why it exists
Keeper takes the 60 s median Kuru mid	up to ~60 s	a few blocks of fake prices do not move a median
Keeper pushes every 30 s	up to ~30 s	gas cost against freshness
Index speed limit $v_{\max} = 0.03$ per minute	$M / v_{\max}$ for a move M	a stolen keeper key can only drag the price slowly
Slow index I_S (10-minute average) must agree	several minutes	short spikes, real or fake, liquidate nobody
Liquidator scans every 2 s; 0.3 s blocks	~3 s	—

The build plan sizes the path buffer for $h = 10$ minutes. §5.2 shows the real delay for large moves is 10–27 minutes and recommends calibrating $h \approx 20$ minutes.

4.2 Buffers (Eq-5, Eq-6)

Closing x units into a book with half-spread s_b and depth density ρ walks the price by x/ρ, so the average execution cost per unit is $\frac{1}{x}\int_0^x \left(s_b + \frac{y}{\rho}\right) dy = s_b + \frac{x}{2\rho}$:

$$e(x) = \underbrace{k\sigma\sqrt{h}}_{\text{path}} + \underbrace{s_b + \frac{x}{2\rho}}_{\text{execution}}, \qquad J(T) = \begin{cases} J_0 & T > T_1 \\ J_1 & T_2 < T \le T_1 \\ J_2 & T_3 < T \le T_2 \\ 1 & T \le T_3 \quad \text{(final tier)} \end{cases}$$

Defaults: $k\sigma\sqrt{h} = 0.03$, $s_b = 0.005$, $x/2\rho = 0.001$, so $e = 0.036$; $J_0, J_1, J_2 = 0.05,\ 0.10,\ 0.20$; $T_1, T_2, T_3 = 7\text{ d},\ 48\text{ h},\ 6\text{ h}$.

4.3 Margin levels and maximum leverage (Eq-7 – Eq-9)

$$MM = x\,\min\left(w(q),\ J(T) + e(x)\right)$$

$$IM = x\,\min\left(c\,w(q),\ \max\left(\frac{w(q)}{L_{\text{cap}}},\ J(T - T_g) + e(x) + u\,w(q)\right)\right), \qquad c = \begin{cases} 1+u & T > T_3 \\ 1 & T \le T_3 \end{cases}$$

$$L_{\max} = \frac{W}{IM} = \max\left(\frac{1}{c},\ \min\left(L_{\text{cap}},\ \frac{w}{J(T - T_g) + e + u\,w}\right)\right)$$

IM looks ahead to the next tier's buffer from $T_g = 30$ minutes before each step, so new positions already meet the coming requirement.

INV-5 for $T > T_3$: $IM - MM \ge u\,x\,w$. J is non-increasing in T, so $J(T - T_g) \ge J(T)$. In the cap branch $IM = (1+u)xw \ge MM + uxw$ since $MM \le xw$. Otherwise $IM \ge x\big(J(T - T_g) + e + uw\big) \ge x\big(J(T) +$

$e\big) + uxw \geq MM + uxw$. ∎

Final tier ($T \leq T_3$). $J = 1$ gives $MM = IM = W$, so by §3.4 an account is healthy if and only if $C \geq x\,w(p_0)$, with no price involved (INV-1).

The headroom $u\,x\,w$ lets a fully collateralised position pay funding for $T_{\min}$ without becoming liquidatable. It also means $L_{\max} < 1$ is possible near the edges; the UI shows that as "no leverage available".

Position	w	$J_0 + e + u\,w$	$w/(J_0 + e + u\,w)$	Native (cap 3x)	Mirrored (cap 5x)
Long at 0.50	0.50	0.111	4.5	3x	4.5x
Long at 0.10	0.10	0.091	1.1	1.1x	1.1x
Long at 0.05	0.05	0.089	below 1	0.95x	0.95x
Short at 0.10	0.90	0.131	6.9	3x	5x

At $p = 0.5$ in a native market, maximum leverage by tier is 3x (tier 0), 3x (tier 1), 1.9x (tier 2) and 1x (final tier).

4.4 Checking the tiers against the diffusion model

With $k = 3$, $p = \frac{1}{2}$, $h = 10$ minutes, compare the diffusion requirement $k\phi(0)\sqrt{h/T}$ at the **end** of each tier, its worst point, with the plan's $J + e$:

Tier ends at T	Diffusion needs	Plan's $J + e$	Surplus = jump budget
7 d	0.038	0.086	0.048
48 h	0.071	0.136	0.065
6 h	0.200	0.236	0.036

The tiers cover diffusion everywhere, and the surplus pays for jumps. Diffusion alone would require full collateral only from $T^* = h\left(k\phi(0)/p\right)^2 \approx 57$ minutes; $T_3 = 6$ h adds about five hours of jump protection. Near the edges the diffusion term shrinks ($\phi(\Phi^{-1}(0.05)) = 0.103$) but jump size does not, because a 0.05 longshot collapsing to 0 is one headline. So J stays absolute. **Recommendation:** let calibration C-1 set J per (tier, price decile).

4.5 Why announced steps instead of a smooth curve

1. **Predictable:** traders see "maximum leverage drops to 1.9x at 48 h" with a countdown.
2. **Time to prepare:** each step is announced $T_g = 30$ minutes ahead through the IM look-ahead.
3. **Fair:** a position pushed under margin by the step itself is deleveraged without penalty (§7).
4. **Simple:** four constants instead of a curve evaluated on every trade.

The two-branch model (§2.5) shows the true requirement is a cliff anyway, and a scheduled cliff beats a surprise one. The continuous version, $MM^{\text{cont}} = x \min\left(w,\ k\phi(z)\sqrt{h/T} + J_{\text{hazard}} + s_b + \frac{x}{2\rho}\right)$, is kept for the paper only.

5. Index, mark and manipulation

> **IN PLAIN WORDS**
> The contract needs a trustworthy price for margin and liquidation. It takes the spot price of the YES token on Kuru, smooths it and limits how fast it can move, so pushing it around is slow and expensive. The perp book's own price, the mark, only tightens checks and sets funding; it can never trigger a liquidation.

5.1 Index (Eq-13)

The keeper samples m_k, the median Kuru mid over the last 60 s of blocks, and pushes it every $\Delta_{\text{poke}} = 30$ s. The chain never reads Kuru directly.

$$\begin{aligned} \Delta &= \min\left(t - t_{\text{idx}},\ \Delta_{\max}\right) \\ I &\leftarrow \text{clamp}\left(I + \text{clamp}(m_k - I,\ \pm v_{\max}\Delta),\ 0.001,\ 0.999\right) \\ I_S &\leftarrow I_S + \tfrac{\Delta}{\Delta+\tau_S}\,(I - I_S) \end{aligned}$$

So $|I(t_2) - I(t_1)| \le v_{\max}(t_2 - t_1)$ (INV-6). A stale index ($t - t_{\text{idx}} > S_{\max}$) blocks new exposure and pauses funding; divergence $|I - I_S| > \delta_I$ switches the market to ReduceOnly. **Change:** `pushIndex` also stores the raw sample m_k, used only to tighten withdrawals (§7.3).

5.2 How long the index lags a genuine move

In continuous time $\dot{I}_S = (I - I_S)/\tau_S$. For a genuine move of size M, I ramps at $v_{\max}$ for $t_1 = M/v_{\max}$:

$$\begin{aligned} I_S(t) &= v_{\max}\left(t - \tau_S(1 - e^{-t/\tau_S})\right) && (t \le t_1) \\ M - I_S(t) &= \left(M - I_S(t_1)\right) e^{-(t-t_1)/\tau_S} && (t > t_1) \end{aligned}$$

During a sustained ramp I_S trails I by up to $v_{\max}\tau_S = 0.30$. For $M = 0.20$ with defaults ($v_{\max} = 0.03$ per minute, $\tau_S = 10$ minutes): $t_1 = 6.7$ minutes and $I_S(t_1) = 0.054$; I_S covers half the move at about 10.5 minutes and 90% at about 27 minutes. Because liquidation needs both indices, **the effective liquidation delay for a large move is 10–27 minutes, not 10.** Options:

1. Calibrate $h \approx 20$ minutes, so $k\sigma\sqrt{h}$ grows by $\sqrt{2}$ (0.030 → 0.042). **Default.**
2. Replace the average with a delayed copy $I_S = I(t - \Delta_c)$, whose lag is fixed. Recorded for the paper.
3. Accept the lag and let J absorb the excess.

5.3 Mark (Eq-14)

$$\begin{aligned} b(I) &= \text{clamp}(0.5\,d,\ 0.01,\ 0.05), \qquad d = \min(I,\ 1 - I) \\ q &= \text{clamp}\left(\text{median}(\text{bid}, \text{ask}, I),\ \max(I - b,\ 0.001),\ \min(I + b,\ 0.999)\right) \end{aligned}$$

A best level holding fewer than $D_{\min}$ units counts as missing (bid 0, ask 1), and $|q - I| \le b(I)$ always (INV-10). Example: $I = 0.55$, bid 0.61, ask 0.63 gives median 0.61, band [0.50, 0.60], so $q = 0.60$. The mark sets funding and tightens new-exposure and withdrawal checks; **liquidation never reads it.**

5.4 Manipulation

Model the Kuru book as a uniform depth of D units per unit of price. Moving the mid by Δ buys $D\Delta$ units at an average slippage of $\Delta/2$:

$$\text{Cost}(\Delta) \approx \tfrac{1}{2}D\Delta^2 \ \text{ per book refill}, \qquad \text{Gain}(\Delta) \approx \lambda\, p\, \rho_{\text{liq}}\, \Delta,$$

where λ is the attacker's share of penalties and ρ_{liq} the density of liquidation thresholds near p. Cost is quadratic and gain is linear, so **small nudges onto positions sitting at their threshold are always cheap.** The design therefore shrinks the gain:

Attack	Defence
Push the index to trigger liquidations	60 s median (the fake price must hold for most of a window); speed limit (lag of at least $\Delta/v_{\max}$); both I and I_S must agree; the position goes to the insurance fund at I, so the attacker cannot buy it; the keeper gets half of a 1% penalty
Inflate own equity, then withdraw (the Mango Markets attack)	withdrawals capped at C (no unrealised profit) and checked at the worst of q, I and m_k
A dust order steers the mark	the $D_{\min}$ filter; the mark only tightens checks; the funding rate changes only at index pushes
The Kuru vault's first-depositor price	the keeper median and the OI cap tie exposure to measured depth
A stolen INDEXER key	drift at most $v_{\max}$ per unit time; divergence switches to ReduceOnly; m_k is used only in the conservative direction

OI cap (Eq-17). $OI_{\max} = k_{OI} D_{\text{spot}}$, where D_{spot} is the 10-minute minimum one-side Kuru depth within ±0.05 of mid. It bounds ρ_{liq}/D, the only lever on the linear gain.

The leverage schedule is itself a defence. In the final tier no position can be liquidated, because $MM = W$ is price-free. Manipulating the index is worthless exactly when resolution volatility would make it cheapest.

6. Funding

> **IN PLAIN WORDS**
> If the perp trades above the index, longs pay shorts a small hourly fee, and the reverse when it trades below; this pulls the two prices together. Because positions settle to the real outcome, funding is only needed while leverage is above 1x, and it stops in the final tier.

Holding YES earns nothing, so the fair perp price is the index and funding is pure premium, measured in **absolute** price units (relative premia blow up near 0):

$$\lambda(d) = \text{clamp}\left(\frac{d_z - d}{d_z - \varepsilon},\ 0,\ 1\right), \qquad g(d) = \alpha + \lambda(d)\,\frac{\beta}{\max(\varepsilon, d)}$$

$$f = \text{clamp}\left(g(d)\,(q - I),\ -\bar{f}_-,\ \bar{f}_+\right), \qquad \bar{f}_+ = \min\left(\bar{f}_{\text{abs}},\ \frac{u\,q}{T_{\min}}\right), \quad \bar{f}_- = \min\left(\bar{f}_{\text{abs}},\ \frac{u(1-q)}{T_{\min}}\right)$$

Cap derivation. At open $IM - MM \geq u\,x\,w$, and funding drains $x f$ per hour, so the payer survives $T_{\min}$ if and only if $f \leq u\,w/T_{\min}$. Longs pay when $f > 0$, hence $w = q$ in $\bar{f}_+$. With the defaults, $|f| \leq g_{\max}\,b(I) = 0.0833 \times 0.05 \approx 0.0042$ USDC per unit per hour.

Accrual (Eq-16). There is one market-wide cumulative index F; each account stores its snapshot F_i. On every market touch $F \leftarrow F + f_{\text{last}}\left(\bar{t}(t) - \bar{t}(t_{\text{fund}})\right)$; on every account touch $C_i \leftarrow C_i - n_i(F - F_i)$ and $F_i \leftarrow F$. Only `pushIndex` sets a new rate, so a single order cannot rewrite funding between pushes. Since $\sum n_i = 0$, funding is zero-sum (INV-3), and updating one number instead of every account keeps it constant-cost.

Freeze. From $\tau - T_3$ every trader holds $C \geq x\,w(p_0)$ with equality possible, so any payment could break full collateral. Hence $f \equiv 0$ from $t_{\text{frz}} = \min(\tau - T_3,\ t_{\text{halt}})$, and while the index is stale.

7. Liquidation, deleverage and withdrawals

> **IN PLAIN WORDS**
> An under-margined position is never sold into the order book. The protocol's insurance-fund account takes over just enough of it at the index price, so there is no cascade and nothing cheap for an attacker to buy. If the tier step itself caused the shortfall, there is no penalty.

7.1 Trigger, penalty and size (Eq-10 – Eq-12)

$$H(p) = E(p) - MM(p), \qquad \text{liquidatable} \iff \max\big(H(I),\, H(I_S)\big) < 0, \qquad \text{executed at } I.$$

$$\varphi_{\text{eff}} = \begin{cases} 0 & E(I) \geq MM_{\text{prev}}(I) \quad \text{(the tier step caused it)} \\ \varphi & \text{otherwise} \end{cases} \qquad MM_{\text{prev}} = x\, \min\big(w,\; J_{j-1} + e\big)$$

Moving Δ units to the IF account at I leaves E unchanged (it is valued at I) and removes the penalty $\varphi_{\text{eff}}\, w(I)\, \Delta$. Requiring initial margin on the remainder, with $i = IM(I)/x$:

$$E - \varphi_{\text{eff}}\, w(I)\, \Delta \geq (x - \Delta)\, i \iff \Delta\,\big(i - \varphi_{\text{eff}}\, w\big) \geq x\, i - E$$

$$\Delta^* = \min\left(x,\; \left\lceil \frac{x\, i - E}{i - \varphi_{\text{eff}}\, w(I)} \right\rceil\right)$$

with $\Delta^* = x$ if $i \leq \varphi_{\text{eff}} w$. Using the pre-liquidation i slightly over-liquidates, because the execution buffer falls with size; that is conservative. Test vectors: $x = 1000$, $E = 167$, $i = 0.25$, $\varphi_{\text{eff}} = 0$ gives $\Delta^* = 332$. A short at $I = 0.9$ with $E = 73$, $i = 0.091$, $\varphi = 0.01$ gives $\Delta^* = 18/0.090 = 200$ and a penalty of 0.2.

7.2 Waterfall

1. Penalty: 50% to the keeper, 50% to the insurance fund (IF).
2. Negative equity left after a full transfer: the IF pays it immediately, up to its balance.
3. Any remainder stays in the shortfall aggregate S_Y and is shared through one ratio r at settlement (§8).

Optional IF hedge. An IF long of x taken at I is exactly hedged by minting x complete sets, selling x YES on Kuru and keeping x NO. The cost is $(1 - I)x$ and the residual risk is only the Kuru execution. This is possible only because markets are native.

7.3 Withdrawals (changed)

$$\text{withdraw} \leq \min\Big(C,\; \min_{p \in \{q,\, I,\, m_k\}} \big(E(p) - IM^{+}(p)\big)\Big)$$

$$IM^{+} = \max\big(IM(n + R_{\text{buy}}),\; IM(n - R_{\text{sell}})\big)$$

No withdrawals are allowed while $|I - I_S| > \delta_I$ or $|m_k - I| > \delta_I$.

- m_k **closes the lag hole.** During a genuine jump I lags by up to $M/v_{\max}$. At the stale I a loser still shows spare collateral, and withdrawing it turns the buffer into bad debt. m_k is only used to tighten the check, so a bad m_k can block withdrawals but never enable one.
- IM^{+} **covers resting orders** (R_{buy} and R_{sell} are the account's resting sizes, kept in the account struct). Without it an attacker posts dust orders at the touch, withdraws the collateral behind them, and makes every taker burn `maxFills` on orders that fail their checks.

8. Settlement

> **IN PLAIN WORDS**
> At resolution each account is valued at the outcome. Winners claim. If some losers could not cover their losses and the insurance fund cannot fill the gap, every winner is paid the same percentage, and no claim depends on who claims first.

Shortfall aggregates (Eq-18) are kept incrementally in the single account-write function (subtract the old term, add the new one):

$$S_Y = \sum_i \max\big(0,\ -E_i(Y)\big), \qquad E_i(Y) = C_i + n_i\,(Y - p_{0,i}), \qquad Y \in \{0, \tfrac{1}{2}, 1\}.$$

A trader with $C \geq 0$ contributes only at the outcome where it loses; a negative C (the IF account, or an unabsorbed remainder) contributes to all three. New exposure on side s is blocked while $S_s > \gamma\,\mathrm{IF}$.

Settlement (Eq-19). Once funding is settled for every open account (frzCount = openCount), conservation at $q = Y$ gives $\sum_i E_i(Y) = B_m$, hence

$$\sum_i \max\big(0, E_i(Y)\big) = \sum_i E_i(Y) + \sum_i \max\big(0, -E_i(Y)\big) = B_m + S_Y.$$

$$\mathrm{draw} = \min(S_Y,\ \mathrm{IF}), \qquad r = \frac{B_m + \mathrm{draw}}{B_m + S_Y} \leq 1, \qquad \mathrm{claim}_i = r\ \max\big(0, E_i(Y)\big).$$

Claims sum to exactly $B_m + \mathrm{draw}$ (INV-4), and $r = 1$ whenever the IF covers the shortfall. Example: $B_m = 10{,}000$, $S = 1{,}500$, $\mathrm{IF} = 1{,}000$ gives a draw of 1,000 and $r = 11{,}000/11{,}500$; claims sum to 11,000.

INVALID (Eq-20) is Eq-19 at $Y = \frac{1}{2}$, matching the OutcomeVault's 0.5/0.5 redemption.

9. The order book

> **IN PLAIN WORDS**
> The order book is fully on-chain, inside the core contract. A row of 999 bits says which prices have orders, so the best price is found in one step. Orders at each price wait in line, oldest first. Storage is arranged so that everything one trade touches sits on as few of Monad's 128-slot storage pages as possible, because each new page costs 8,100 gas and each brand-new slot 17,000 more.

9.1 What it must do

Price-time priority: better price first; at the same price, older order first. Four operations, each cheap and bounded:

Operation	What it does	Cost target
Best price	highest bid, lowest ask	constant; runs on every trade and every mark computation
Place	append an order to its price's queue	constant
Cancel	remove an order from anywhere in its queue	constant; makers cancel constantly
Match	walk the best prices, filling the oldest orders first	one step per fill, capped by `maxFills`

9.2 Finding the best price: a bitmap, not a red-black tree

Solady's `RedBlackTreeLib` keeps a sorted set of arbitrary keys; it is the right tool when prices can take billions of values. Each insert or delete walks about $\log_2 n$ nodes and may rotate them, and each node on an untouched page costs 8,100 gas to read. EventPerp's prices are the ticks 0.001 … 0.999, only 999 values, so one bit per tick is enough:

```
tick:  ... 497 498 499 500 | 501 502 503 504 ...
bids:  ...  1   1   0   1  |  0   0   0   0  ...   best bid = highest set bit -> 0.500
asks:  ...  0   0   0   0  |  0   1   1   0  ...   best ask = lowest set bit  -> 0.502
```

- 999 bits fit in 4 words of 256 bits per side: 8 words for both sides, on one storage page.
- Best bid and best ask come from `LibBit.fls` and `LibBit.ffs` per word (Monad also supports the `CLZ` opcode), a few hundred gas in total.
- Adding or removing a price level flips one bit.

Kuru, the spot book on Monad, uses the same overall pattern: a structure over active prices plus a FIFO list per price. With 999 ticks the bitmap is the simplest and cheapest form of it.

9.3 Orders at one price: a doubly linked FIFO list

```
tick 502 (asks)   Level{head=#7, tail=#12, size=900}
                  +--------+   +--------+   +--------+
   head --------> | #7 300 |<->| #9 100 |<->| #12 500| <-------- tail
                  +--------+   +--------+   +--------+
                  oldest: fills first                 newest
```

Place appends at the tail and matching fills from the head. Cancel links the two neighbours together, which is why every order stores `prev`: it makes cancelling from the middle constant-cost. `size` is the total resting at the tick, so the mark's $D_{\min}$ check is a single read.

Lazy fills are ruled out. Some spot books clear a whole level in one step and settle each maker later. A perp cannot: every maker fill must first check that maker's margin and the market stage.

9.4 Monad's storage pricing (MIP-8, live since 2 September 2026)

Charge	Gas	When
Page load	8,000 + 100 base	first read or write of a 128-slot page in a transaction
Warm access	100	any other slot on an already-touched page
Page write	2,800	first write that changes a value on that page in the transaction
State growth	17,000	each zero → non-zero write that raises the page's slot count to a new high in the transaction

A page is 128 consecutive slots (`slot >> 7`). Struct fields and array elements share pages; each mapping key lands on its own page. There are no refunds, fees are charged on the **gas limit**, and a reverting call frame rolls page warmth back. A book operation therefore costs about **(distinct pages touched) × ~11k + (brand-new slots) × 17k**.

9.5 Storage layout: four rules

1. **Co-locate what is read together.** The market's config, state, both bitmaps and the free-slot pointer live in one struct, usually one page. Each tick's bid and ask levels are neighbours (`Level[2][1000]`), so the few ticks one trade touches share a page. Orders live in one dense array indexed by slot, not a mapping, so a market with under ~128 live orders keeps them on one or two pages. Each (trader, market) account is one struct.
2. **Never let a slot that empties and refills return to zero.** Levels empty and refill constantly, and zeroing them would charge 17,000 on every refill. A `used` flag keeps level slots non-zero, and one always-set bit per bitmap word does the same for the bitmaps (tick k maps to word $\lfloor (k-1)/250 \rfloor$, bit $(k-1) \bmod 250$; bit 255 is the sentinel).
3. **Recycle order slots.** A dead order stays non-zero (a tombstone) and its slot goes on a free-list stack threaded through `next` . New orders pop from it and overwrite a non-zero slot, so they pay no 17,000. To stop a stale cancel from hitting a new order in a reused slot, the slot carries an 8-bit generation that is part of the public id (`gen << 24 | slot`). This still fits the frozen `uint32` interface and allows 16.7 million live orders per market.
4. **One contract, no external calls in the match loop.** A first call to another contract costs 10,100 gas on Monad, and a revert discards page warmth. Book and Clearing talk through internal functions. Accounts are keyed by `uint32` trader id and `Fill` events carry ids, so matching never looks up addresses.

```
struct Level {            // 1 slot
    uint32 head;          // oldest order slot
    uint32 tail;          // newest order slot
    uint96 size;          // total resting units at this tick
    bool   used;          // stays true after first use: the slot never returns to zero
}

struct Order {            // 1 slot, 224 bits
    uint32 owner;         // trader id
    uint96 size;          // remaining units; 0 = dead
    uint32 next;          // next in level; while dead: next free slot
    uint32 prev;
    uint16 tick;          // 1..999
    uint8  flags;         // BUY | REDUCE_ONLY | LIVE
    uint8  gen;           // bumped on every reuse; part of the public id
}

struct Book {             // inside the per-market struct
    uint256[4] bids;      // tick k -> word (k-1)/250, bit (k-1)%250; bit 255 = sentinel
    uint256[4] asks;
    uint32 freeHead;      // top of the free-slot stack
    Level[2][1000] levels;
    Order[] orders;       // index = slot; slot 0 = null
}
```

9.6 The matching loop

Setup, once per transaction: copy a snapshot into memory (index, mark, stage, tier, gates, open interest, shortfall aggregates) and load the taker's account, settling its funding. Every check in the transaction uses this snapshot.

```
while (want > 0 && steps < maxFills) {
    k = bestAsk();
    if (k == NONE || k > limit) break;
    Level storage lv = levels[k][ASK];
    for (s = lv.head; s != 0 && want > 0 && steps < maxFills; ) {
        Order storage o = orders[s];  steps++;  next = o.next;
        if (o.owner == takerId) { _remove(s, SELF_TRADE); s = next; continue; }
        // Clearing checks the maker (stage, reduce-only, IM at q and I, stress), updates it
        f = _makerFill(ctx, o.owner, SELL, k, min(want, o.size), o.flags);
        if (f == 0) { _remove(s, FAILED_CHECK); s = next; continue; }
        _takerFill(ctx, BUY, k, f);                  // taker side: memory only
        emit Fill(m, id(s), o.owner, takerId, k, f);
        want -= f;  lv.size -= f;
        if (f == o.size) { _remove(s, FILLED); s = next; }  // unlink, recycle, clear empty bit
        else o.size -= f;      // partial: stays first; cancel the rest if reduce-only clipped
    }
}
_takerDone(ctx);   // taker IM, OI and stress checks, else revert; ONE write of taker + totals
if (type == LIMIT && want > 0 && !crosses(limit)) _rest(BID, limit, want);
```

- Every step counts toward `maxFills`: fills, self-trade cancels and failed maker checks. Gas stays bounded even if an attacker stacks the touch with orders that will fail.
- Monad charges the gas limit, so `maxFills` is effectively the taker's gas budget. The UI defaults to about 8.
- The taker's account and the market aggregates are written once, at the end; each maker account once per fill.

9.7 Order types and edge-case rules

LIMIT matches and then rests the remainder; IOC matches and then drops it; POST_ONLY never matches. Any order may be reduce-only: it can only shrink the position, is clipped at fill time and is cancelled at zero. A self-trade cancels the resting (maker) order. Three rules the build plan does not yet state:

1. **No crossed remainder.** If `maxFills` runs out while the book still crosses the taker's limit, the LIMIT remainder is dropped, not rested. Otherwise bid ≥ ask, which breaks INV-8 and the mark's median.
2. **Idempotent cancels.** Inside `batch`, cancelling an order that already filled or was already cancelled does nothing. Otherwise any fill that lands between a bot's read and its requote reverts the whole requote. Kuru's `batchCancelOrders` behaves this way.
3. **A crossing post-only order inside `batch` is skipped** (it returns id 0) instead of reverting the batch.

9.8 The seam between Book (R1) and Clearing (R2)

```
abstract contract Book {
    // Clearing overrides these hooks.
    // Book never touches accounts; Clearing never touches the book.

    // returns 0 = failed check (cancel the order); less than size = reduce-only clip
    function _makerFill(Ctx memory c, uint32 maker, bool makerBuys, uint16 tick,
                        uint96 size, uint8 flags) internal virtual returns (uint96 filled);

    // taker side, memory only
    function _takerFill(Ctx memory c, bool takerBuys, uint16 tick, uint96 size)
        internal virtual;

    // final taker checks and the single account write; reverts on failure
    function _takerDone(Ctx memory c) internal virtual;

    // resting-order margin (IM+), tracked in the account's R_buy and R_sell
    function _onRest(uint32 trader, bool isBuy, uint16 tick, uint96 size) internal virtual;
    function _onUnrest(uint32 trader, bool isBuy, uint96 size) internal virtual;
}
```

A test harness that overrides these with trivial versions lets R1 finish and test the book before R2's accounts land. The same hooks would serve a hybrid settlement path (§10.5), because the risk engine never depends on where matching happens.

9.9 Gas estimates (MIP-8 prices, to be measured)

Operation	Pages touched	Estimate
Place a resting order	sender id, market, level, order, account	~75–95k
Cancel and replace in one `batch`	the same	~80–95k
Taker order with one fill	plus maker account, taker account, IF	~90–100k
Each extra fill, same maker	nothing new	+3–4k
Each extra fill, new maker	plus that maker's account page	+~14k

The estimates include the 21,000 base transaction cost. The build plan's budget of under 100k for cancel-and-replace holds with this layout; with a fresh slot per order it does not.

9.10 What not to build

Idea	Why not
Red-black tree or heap for prices	built for unbounded price spaces; more reads and writes per order
Lazy-fill queues	skip the per-maker margin and stage checks a perp needs
Reusing Kuru's book for the perp	a spot book that moves tokens; no margin hooks; source not public
One contract per market	10,100 gas per cold cross-contract call, plus reentrancy surface
An on-chain depth getter	Envio rebuilds the book from `OrderPlaced`, `OrderCancelled` and `Fill`
A cached best price	the bitmap words already sit on the warm market page

9.11 Tests and build order

1. Structs, bitmap helpers and `bestBidAsk`, with unit tests.
2. Rest and cancel with the free list and generation ids; INV-8 as a Foundry invariant (bit set ⇔ level non-empty; level size = sum of its orders; links agree; best bid < best ask).
3. The match loop against the harness hooks: all order types, self-trade, `maxFills`, the crossing rule.
4. `batch`: cancels first and idempotent; crossing post-only orders skipped.
5. A differential test: the same random operations fed into Book and into a naive reference book written in the test must produce identical fills.
6. Gas snapshots with `network = "monad"`, after first confirming that Foundry's Monad mode prices storage with MIP-8. If it still prices like Ethereum, the numbers will be badly off.
7. Swap the harness for R2's Clearing hooks.

10. Latency and the hybrid question

> **IN PLAIN WORDS**
> A hybrid book matches orders on a fast off-chain server and settles the trades on-chain. On Monad it would make order confirmations and cancels feel several times faster and make quoting free for market makers. But trades would still become final at the same speed as today, a trusted operator would decide ordering, and building it would take one to two developer-weeks the team does not have before 13 October. Recommendation: stay fully on-chain for the hackathon, use Monad's early (speculative) data to make the UI feel fast, and keep the hybrid below as a ready design for later.

10.1 Where latency comes from on Monad

Block state	Time after proposal	Meaning	RPC tag
Proposed	0	the leader proposed it; executed speculatively within milliseconds	`latest`
Voted	~300 ms (1 slot)	speculatively final; reverts only if the leader equivocated	`safe`
Finalized	~600 ms (2 slots)	cannot be reverted without a hard fork	`finalized`
Verified	~1.5 s (derived: 3-block execution delay, then 2 to finalize)	state root agreed	—

Standard `newHeads` and `logs` WebSocket subscriptions already fire at **Proposed**; `monadNewHeads` and `monadLogs` add commit-state updates. Nearly all proposals finalize, but a proposal can be abandoned, so apps track blocks by `blockId` and discard work if a different block finalizes at that height. The Execution Events SDK is faster still (microseconds, read from shared memory) but must run on the same host as a Monad node, in C, C++ or Rust.

For a user of the fully on-chain book, propagation to the leader (roughly 50–150 ms) plus waiting for the next slot (0–300 ms) means **a placement, cancel or fill is visible about 0.1–0.45 s after sending**, and final about 0.6 s later.

10.2 The architectures

	A. Fully on-chain (current)	B. Operator hybrid	C. Validator books
Example	Kuru on Monad; Hyperliquid on its own chain	Polymarket's CTF Exchange	dYdX v4
Matching	in the contract	an off-chain operator, in memory	each validator's in-memory book; the proposer's matches go into the block
Settlement	in the same transaction	the operator submits matched, signed orders	in the block
Fits EventPerp?	yes	yes	no: needs its own chain

10.3 Latency: A against B

Moment	A. Fully on-chain	B. Hybrid
Order or cancel acknowledged	~0.1–0.45 s (at Proposed)	~0.02–0.1 s (round trip to the operator; co-located bots under 5 ms)
Fill known to the trader	~0.1–0.45 s	~0.02–0.1 s, but only a promise until settled
Fill on-chain (Proposed)	the same moment	plus the next slot after the match: ~0.1–0.45 s
Final	plus ~0.6 s	plus ~0.6 s
Maker gas per requote	~80–95k	0

The hybrid speeds up acknowledgement and cancels, not settlement. Its real wins are free, instant requotes for market makers (tighter spreads, and no MON budget risk for the mm-bot) and instant-feeling fills in the UI.

10.4 What the hybrid costs EventPerp

Cost	Detail
Trust	Funds stay safe (signed orders, on-chain checks), but the operator decides ordering, can front-run or censor, and trading stops while it is down. Polymarket's operator "can match orders and enforce their ordering" but "cannot set prices or execute trades that users did not authorize"
Mark price	Eq-14 reads the perp book's bid and ask on-chain, and a hybrid has no on-chain book. Either the operator posts its best bid and offer (trusted, but the band already limits damage to $\pm b(I)$ and liquidation never reads the mark) or the mark uses an average of settled fill prices
Soft fills can fail	The operator pre-checks margin on a shadow copy. If the chain disagrees at settlement (an index push landed first), the fill is rolled back, so settlement must skip failed matches instead of reverting the batch

Cost	Detail
Replay protection	Each order's filled amount is stored on-chain: ~17,000 gas of state growth per new order that fills
Signatures	`ecrecover` costs 6,000 gas on Monad (twice Ethereum's), two per match unless session keys are registered
New components	matching service, WebSocket order API, persistence, settlement batcher with reconciliation, EIP-712 signing in the UI, session keys, and changes to the mm-bot, Envio and the dashboard
Schedule	about one to two developer-weeks plus integration, landing on M1 (2 Oct) and M2 (7 Oct)
Pitch (judgement)	a fully on-chain perp CLOB is the stronger story at a Monad hackathon than an operator like Polymarket's

10.5 The hybrid design (phase 2, if built)

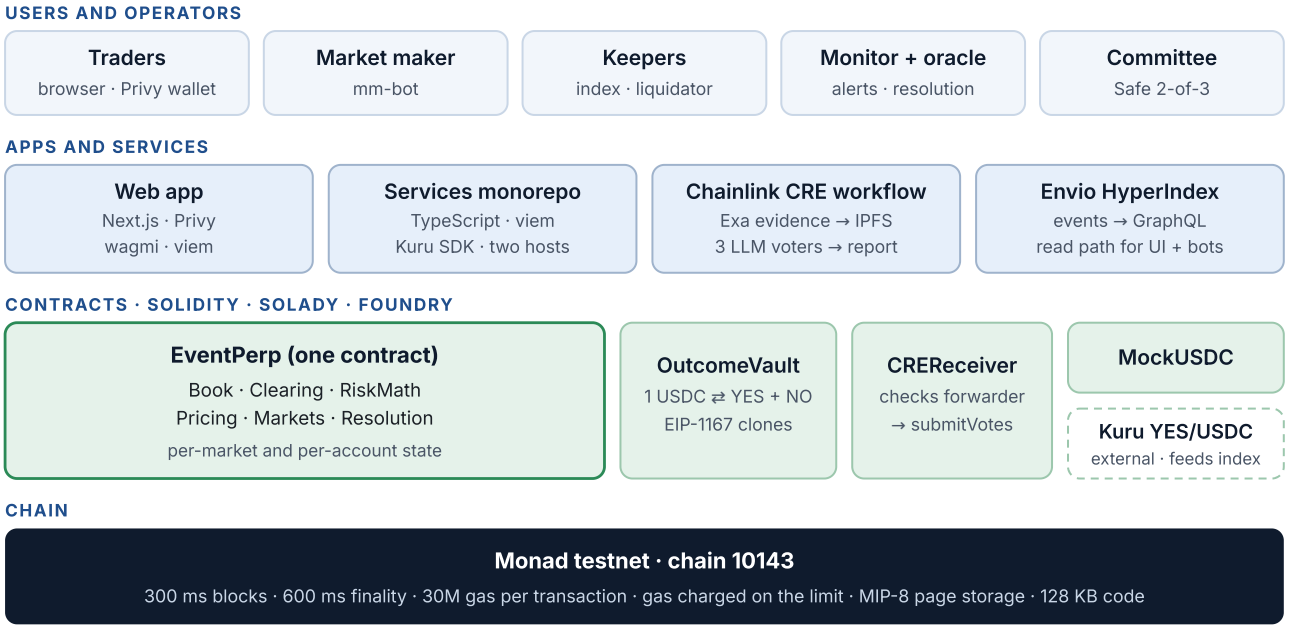
1. **Signed orders (EIP-712):** `Order{market, trader, isBuy, tick, size, flags, expiry, salt}`, signed by the trader or by a registered session key that can trade but not withdraw and that expires. Privy's embedded wallet signs without pop-ups.
2. **Operator:** an in-memory book per market with the same price-time rules; a shadow copy of the risk state (a TypeScript port of RiskMath, tested against the Solidity vectors); and a published, signed sequence log of every accepted action, so misordering is provable after the fact. An optional 250 ms taker delay protects makers; Polymarket uses one on selected markets.
3. **Settlement entry** `settle(Match[])`, operator-only: for each match, verify the signatures, the expiry and that the cumulative fill is at most the order size; execute at the maker's tick within the taker's limit; then call the **same** `_makerFill`, `_takerFill` and `_takerDone` hooks as the on-chain book (§9.8). A failed match is skipped and reported, not reverted.
4. **Cancels:** an instant soft cancel at the operator, plus an on-chain kill switch, `bumpEpoch()`, that invalidates all of a trader's older orders in one slot write.
5. **Escape hatches:** deposits, top-ups and withdrawals are always direct on-chain. Liquidation (index-based, never uses the book) and resolution (pull claims) run without the operator. If the operator stalls, positions stay solvent and traders can add collateral; they cannot trade until it returns.
6. **Mark:** the operator's best bid and offer, clamped to $I \pm b(I)$ as today.

10.6 Cheap latency wins for the on-chain book (do now)

- The UI and bots subscribe to `logs` at `latest` (Proposed) and show fills as **pending → final**, tracking each `blockId` until Finalized. Fills appear about 0.1–0.45 s after sending instead of after finality.
- Privy's embedded wallets sign without a pop-up; keep one signature per action.
- Bots set explicit gas limits (fees are charged on the limit) and pay a higher priority fee on cancels than on takes.
- Bots run close to their RPC endpoint. The Execution Events SDK is worth it only if the team runs its own Monad node.

> **VERDICT**
>
> Yes, a hybrid is feasible, and EventPerp's risk engine makes it unusually easy: margin, liquidation, funding and settlement never depend on where orders are matched, so only the book and the mark's input change. But it cuts acknowledgement and cancel latency (from ~0.1–0.45 s to ~0.02–0.1 s), not settlement, and it adds a trusted operator and one to two developer-weeks. **For the 13 October submission, stay fully on-chain and ship the §10.6 wins. Revisit the hybrid after the hackathon if market makers need free, instant requotes.**

11. Technology stack



The stack as layers: who uses EventPerp, the apps and services they run, the contracts, and the chain underneath. The dashed box is Kuru's contract, not ours.

11.1 Chain and network

Item	Choice	Notes
Chain	Monad testnet (chain id 10143); mainnet is 143	300 ms blocks, 600 ms finality, 30M gas per transaction, 150M per block
Gas model	charged on the gas limit, not gas used	bots and the UI set explicit, measured limits
Storage pricing	MIP-8 page-based storage	shapes the order book layout (§9.4)
Contract size	128 KB limit (Ethereum: 24 KB)	one core contract fits
RPC	Alchemy (bounty) as primary, public Monad RPC as fallback	WebSocket for real-time data
Real-time data	`logs` and `newHeads` at Proposed; `monadLogs` for commit states	§10.6

11.2 Smart contracts (R1–R4)

Item	Choice	Notes
Language	Solidity 0.8.x	WAD fixed point; every rounding against the trader
Framework	Foundry ≥ 1.8 with `network = "monad"`	tests, fuzzing and invariants, `forge snapshot --check`, `forge build --sizes`, `forge script` deploys
Library	Solady	`LibBit` (bitmaps), `SafeTransferLib` (USDC), `FixedPointMathLib` (WAD, rounding direction), `LibClone` (EIP-1167 token clones), `OwnableRoles` (roles), `ERC20` (outcome tokens, MockUSDC); `EIP712` and `SignatureCheckerLib` only for the hybrid

Item	Choice	Notes
`EventPerp.sol`	one deployment composing `Book`, `Clearing`, `RiskMath`, `Pricing`, `Markets`, `Resolution`	per-market and per-account state; internal hooks; no external calls in hot paths
`OutcomeVault.sol` + `OutcomeToken.sol`	complete sets, 1 USDC ⇄ 1 YES + 1 NO; token clones per market; INVALID redeems 0.5/0.5	native markets
`CREReceiver.sol`	Chainlink receiver: checks the forwarder (settable, since simulation and production differ) and the workflow id, then calls `submitVotes`	oracle path
`MockUSDC.sol`	6 decimals, rate-limited faucet	testnet only
Verification	contracts verified on the Monad testnet explorer; addresses in the README	R1-7

11.3 Oracle and resolution (R4)

Item	Choice	Notes
Workflow runtime	Chainlink CRE, TypeScript SDK ≥ 1.19, CLI ≥ 1.30	HTTP trigger with the evidence bundle's IPFS CID; DON-signed report to `CREReceiver`
Evidence	Exa search over the sources committed at market creation; bundle ≤ 100 KB	hash on-chain; file on IPFS and in repo artifacts
Voters	three LLMs from different providers; temperature 0 where allowed; each voter's answer cached across DON nodes	unanimity and confidence ≥ θ enforced on-chain
Disputes	bonded challenge window (2 h; 3 min in demos); committee = Safe 2-of-3	the watcher pages the committee
Monitor	Jev (if the replication passes on 5 Oct) plus a text-LLM second signal	alert → ReduceOnly; known outcome → Halted
Fallback	the same payload signed by the PROPOSER key	if CRE is unavailable

11.4 Off-chain services (R1, R3, R4)

Item	Choice	Notes
Repository	one TypeScript monorepo (pnpm workspaces)	a shared `abis` package generated from Foundry artifacts with `@wagmi/cli`
Runtime	Node.js LTS	
Chain client	viem	typed calls, multicall, WebSocket subscriptions
Spot	Kuru SDK plus RPC	Kuru's UI and WebSocket are mainnet-only
index-keeper	reads Kuru `bestBidAsk()` every block; pushes the 60 s median every 30 s	INDEXER key
mm-bot	perp quotes through one `batch` per tick move; mints sets and quotes YES/USDC on Kuru	pulls quotes on divergence above 0.02, alerts or staleness

Item	Choice	Notes
liquidator	multicalls `health` every 2 s and calls `liquidate`; funding sweeps at $\tau - T_3$ and on halt	permissionless
monitor, oracle, watcher	§11.3	MONITOR and PROPOSER keys
Keys	one key per role, never shared; ADMIN and COMMITTEE on the Safe	
Hosting	two hosts from different providers; containers with restart policies	X3: keeper downtime
Research	Python with pandas, NumPy and SciPy on the PMXT v2 Polymarket archive (1-minute bars)	calibration C-1 to C-8

11.5 Data, frontend and operations (R1, R5)

Item	Choice	Notes
Indexer	Envio HyperIndex	orders, fills, positions, funding, liquidations, stages, votes, claims; GraphQL is the only read path for the UI and bots besides view calls
Web app	Next.js with TypeScript; Privy (login, embedded wallets); wagmi and viem	trade page (book, IOC-default entry, positions, leverage on the W basis, stage badge, tier countdown), risk dashboard, resolution page
Governance	Safe 2-of-3 on Monad testnet	ADMIN, COMMITTEE
CI	GitHub Actions	`forge test` (unit, fuzz, invariants), `forge snapshot --check`, `forge build --sizes`, TypeScript typecheck and tests, Envio codegen
Hybrid layer (only if §10 changes)	operator service (TypeScript or Rust), WebSocket order API, Postgres for order state, EIP-712 signing and session keys	§10.5

11.6 Who calls what

From	To	Calls
Web app	EventPerp	`deposit`, `withdraw`, `placeOrder`, `cancel`, `batch`, `claim`
index-keeper	Kuru, then EventPerp	reads `bestBidAsk()`; calls `pushIndex`
mm-bot	EventPerp, Kuru, OutcomeVault	perp quotes via `batch`; spot quotes; mints complete sets
liquidator	EventPerp	`liquidate`, `settleFunding`
monitor	EventPerp	`alert`, `halt`
oracle	CRE DON, then CREReceiver, then EventPerp	`submitVotes`
Safe	EventPerp	`createMarket`, `setRiskParams`, `committeeResolve`, `unhalt`, `clearAlert`
EventPerp	Envio, then the web app and bots	events

12. Changes to the build plan and open decisions

12.1 Changes

#	Change	Build plan	Why
1	`pushIndex` stores m_k; withdrawals are checked at the worst of q, I and m_k and blocked while the indices diverge	§4.4, Eq-13	losers could withdraw buffer during index lag (§5.2, §7.3)
2	Withdrawal IM includes resting orders (R_{buy}, R_{sell} in the account)	§4.3, §4.4	dust orders could make takers burn `maxFills`
3	C-2 calibrates h to the two-index delay (~20 minutes)	§3, §11	the slow average lags a ramp by up to $v_{\max}\tau_S$
4	C-1 sets J per (tier, price decile)	§3, §11	jump size scales with w near the edges
5	A LIMIT remainder is dropped if the book still crosses when `maxFills` runs out	§4.4	a resting remainder would cross the book (INV-8)
6	Cancels are idempotent inside `batch`	§4.5	a stale cancel must not revert a requote
7	A crossing post-only order inside `batch` is skipped (id 0)	§4.4	same reason
8	Order slots are recycled; the `uint32` order id packs an 8-bit generation with the slot	§4.3	saves ~17,000 gas per placed order under MIP-8
9	Level slots and bitmap words never return to zero (sentinels)	§4.3	a refilled level would pay 17,000
10	Accounts are keyed by `uint32` trader id; `Fill` events carry ids	§4.3, §4.5	no address lookups inside matching
11	The UI defaults to `maxFills` ≈ 8	§4.7, R5-3	fees are charged on the gas limit
12	Day-1 check: Foundry's Monad mode prices storage with MIP-8	§10	gas budgets are meaningless otherwise
13	New tests: a 0.20 jump with a withdrawal during the ramp (refused); resting orders reduce withdrawable collateral	§6	cover changes 1 and 2
14	The UI and bots read fills at `latest` and track them to Finalized	§4.7	perceived latency ~0.1–0.45 s (§10.6)

For the paper: Proposition 1's invariant is enforced **from the final tier**, not at the halt; and "10x is not offered" rests on jumps (one 5-cent jump wipes it out), not on hourly volatility.

12.2 Open decisions

#	Decision	Recommendation	Owner
1	Recycle order slots with a generation inside the `uint32` id	yes: saves ~17k gas per order and cuts the mm-bot's MON cost	R1
2	Sizes in whole units	yes: each fill's notional is exactly size × tick in 0.001 USDC, so the book never rounds; Eq-12's ceiling and $D_{\min} = 250$ units already assume it	R2
3	Slow index: average with $h \approx 20$ minutes, or a delayed copy	average with recalibrated h now; delayed copy in the paper	R3

#	Decision	Recommendation	Owner
4	Insurance-fund hedge through the OutcomeVault and Kuru	stretch goal	R3
5	Hybrid order book	not for 13 October; revisit after the hackathon	team

13. Limitations

1. **Jumps are bounded, not removed:** the stress gate, the insurance fund and the ratio r handle them.
2. **Compression pre-empts some winners;** calibration C-7 measures how many.
3. **Markets that can resolve at any moment** break the time-to-resolution model, so only scheduled categories are listed.
4. **Index lag trades manipulation resistance for liquidation delay** (§5.2). The trade-off can be moved, not removed.
5. **Thin books early on:** a small D means small OI caps, and on testnet the Kuru mid is mostly the mm-bot's own quotes.
6. **Trust:** the team holds the INDEXER, MONITOR and PROPOSER keys; the committee judges its own proposals; the LLM voters' errors are correlated (0.53–0.69).
7. **Calibration** uses one week of data, 51% of it sports.
8. **Funding near the edges** is capped, so its pull is not uniform there.
9. **Testnet only:** MockUSDC, no real money.

14. Sources

- **Build plan:** `roadmap.md`, 27 September 2026 (team document).
- **Monad:** Differences from Ethereum · Opcode pricing (MIP-8 storage costs) · MIP-8 specification · MIP-8 summary · Gas pricing · Changelog (CLZ, MONAD_TEN) · MonadBFT · Block states · Speculative real-time data · WebSocket subscriptions · Asynchronous execution · Execution Events SDK
- **Kuru:** OrderBook contract · Router · Backstop AMM
- **Polymarket:** Order lifecycle · Trading overview · CTF Exchange v2
- **dYdX v4:** Architecture overview · Onboarding FAQs
- **Solady:** RedBlackTreeLib · LibBit